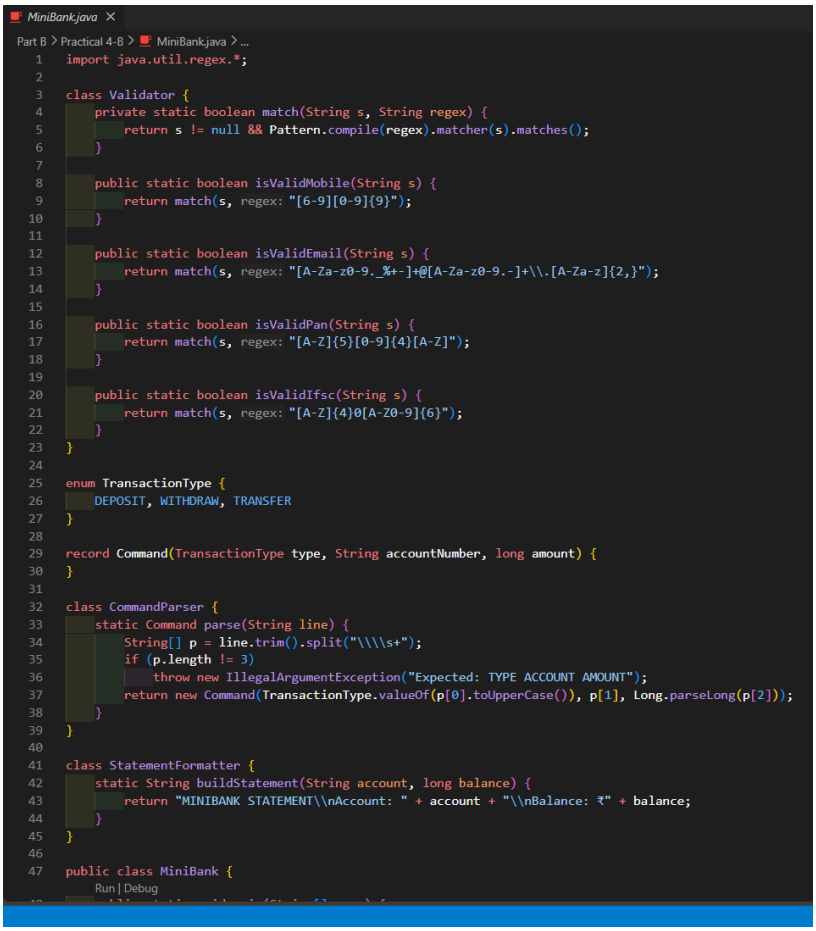
PRACTICAL 4-B

Object Oriented Programming

AIM:

To implement MiniBank input validation and command parsing using regular expressions, Pattern, Matcher, records and StringBuilder.

CODE SCREENSHOT:



```java
import java.util.regex.*;

class Validator {
    private static boolean match(String s, String regex) {
        return s != null && Pattern.compile(regex).matcher(s).matches();
    }

    public static boolean isValidMobile(String s) {
        return match(s, regex: "[6-9][0-9]{9}");
    }

    public static boolean isValidEmail(String s) {
        return match(s, regex: "[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}");
    }

    public static boolean isValidPan(String s) {
        return match(s, regex: "[A-Z]{5}[0-9]{4}[A-Z]");
    }

    public static boolean isValidIfsc(String s) {
        return match(s, regex: "[A-Z]{4}0[A-Z0-9]{6}");
    }
}

enum TransactionType {
    DEPOSIT, WITHDRAW, TRANSFER
}

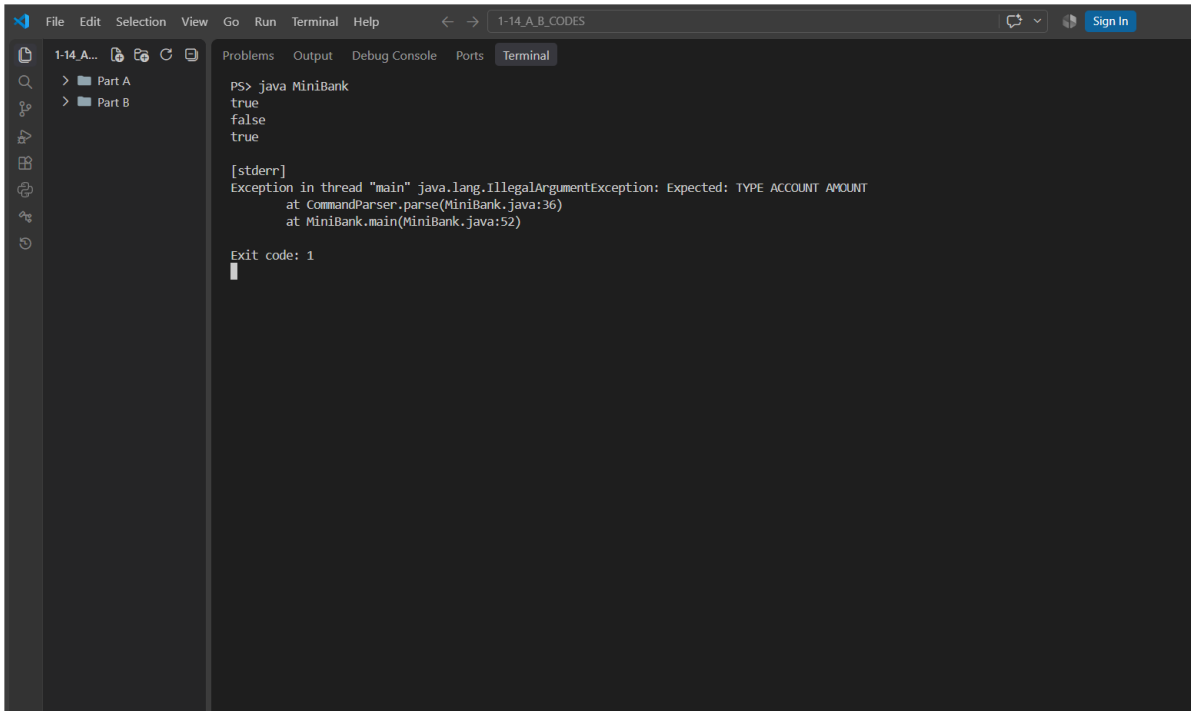
record Command(TransactionType type, String accountNumber, long amount) {
}

class CommandParser {
    static Command parse(String line) {
        String[] p = line.trim().split("\\s+");
        if (p.length != 3)
            throw new IllegalArgumentException("Expected: TYPE ACCOUNT AMOUNT");
        return new Command(TransactionType.valueOf(p[0].toUpperCase()), p[1], Long.parseLong(p[2]));
    }
}

class StatementFormatter {
    static String buildStatement(String account, long balance) {
        return "MINIBANK STATEMENT\\nAccount: " + account + "\\nBalance: ₹" + balance;
    }
}

public class MiniBank {
    Run | Debug
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can validate and parse text input reliably using strings and regular expressions.